

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 5 : SLO – 2 | Graph Implementation Using Array — Deletion of Node and Edge

■ I. Fill in the Blanks

- Deleting a node in an adjacency matrix requires removing the corresponding ROW and COLUMN.
- In an adjacency list, deleting a node requires removing its list and all occurrences of it in other vertices' LISTS.
- Deleting an edge in an adjacency matrix involves setting 0 in the corresponding cell.
- In an undirected graph, deleting an edge between A and B requires updating both $MATRIX[A][B]$ and $MATRIX[B][A]$.
- Deleting a node will also remove all its EDGES (incident edges).

■ II. True or False

- Deleting a node in an adjacency list is faster than in an adjacency matrix — FALSE (list deletion must scan all other lists; matrix is simpler).
- In an undirected graph, removing an edge requires one operation in the adjacency list — FALSE (two lists updated).
- When deleting a node, its edges remain intact in a simple graph — FALSE.
- In an adjacency matrix, deleting an edge requires updating two cells in an undirected graph — TRUE.

■ III. Pseudocode — Deletion

- Deletion of an Edge (Adjacency List):
 1. Locate the adjacency list of the source vertex.
 2. Remove the DESTINATION vertex from the list.
 3. If the graph is undirected, also remove the source vertex from the adjacency list of the DESTINATION vertex.
- Deletion of a Node (Adjacency List):
 1. Remove the adjacency list of the vertex to be deleted.
 2. For each other vertex, REMOVE (delete) the node to be deleted from its adjacency list.
 3. Update the total number of vertices.

■ IV. Spot the Error

- Mistake: $matrix[u][u] = 0$ clears the diagonal instead of the reverse edge.
- Correct it to: $matrix[u][v] = 0$; $matrix[v][u] = 0$;

■ V. Complete the C Code

- Adjacency Matrix:
- `void deleteEdge(int graph[10][10], int u, int v) {`

- `graph[u][v] = 0;`
- `graph[v][u] = 0; // undirected graph`
- `}`
- Adjacency List:
- `void deleteEdge(struct Node* adjList[], int u, int v) {`
- `removeNode(&adjList[u], v);`
- `// For undirected graph:`
- `removeNode(&adjList[v], u);`
- `}`

■ VI. Predict the Output

- After `deleteEdge(graph, 0, 1)`:
- `0 1 2`
- `0 [ 0 0 1 ]`
- `1 [ 0 0 0 ]`
- `2 [ 1 0 0 ]`